
```

% Ezinne Akabueze, CMOR 220, Spring 2026, Project 3: Roots
% Project2_Roots.m
% This coding program is tasked to compare the efficiency of 1) the
% bisection methods and 2) newton's method in finding a bar's cooling
% rate, each tasked to find the smallest root of the given equation.
% Last modified: 2/24/26
clc;
clear;

disp('Competency Functions')
% Defining all necessary functions for Project 3:

% Function for newton's method that calls the coolfun and coolfundx, and
% estimates the smallest positive root x using newton's method
function [x, iter] = denewt(x1, tol, L) % tol is tolerance, L is bar length,
and X1 is the smallest positive root
    iter = 0;
    x = x1;
    while abs(coolfun(x,L)) > tol
        deriv = coolfundx(x,L);
        if deriv == 0
            disp('Newton method is unable to proceed because derivative is
0.')


---



```

```

% Function that evaluates cool for a given x and L
function [val] = coolfun(x,L) % x represents the smallest positive root,
while L is bar length
    val = sin(x*L) + x*cos(x*L); % original function
end

% Function that evaluates the derivative of cool fun, with respect to x
function val = coolfundx(x,L) % x represents the smallest positive root,
while L is bar length
    val = cos(x*L)*L + cos(x*L) - L*x*sin(x*L); % derivative of original
function
end

% Part 1: Bisection Method for variable bar length
disp('Part 1')
% Define variables
L = 1:.1:4; % Length of the cooling bar varies for figure 1
tol1 = .01;
a = .01; % left endpoint fixed in place
x = zeros(1,length(L)); % preallocate root section

for cnt = 1:length(L)
    % continuously move right endpoint until region is small enough to
    % define root
    b2 = 3/L(cnt); % vary right endpoint in order to ensure you're finding
the smallest room
    [x(cnt),~] = debis(a,b2,tol1,L(cnt));
end

figure(1)
plot(L,x.^2,'x-') % plot cooling rate vs length, label your plot
title('Cooling vs bar length')
xlabel('length, L')
ylabel('decay rate, x^2')

% Part 2: Compare Bisection and Newton's Method for variable tolerance
disp('Part 2')
% Define variables
L1 = 1; % fixed length for figure 2
tol2 = 10.^(1:8)'; % fixed left endpoint for figure 2
a = .01; % fixed left endpoint for figure 2
b = 3; % fixed right endpoint for figure 2
x = (a+b)/2; % find the midpoint
itBis = zeros(1,length(tol2)); % preallocate for iteration vector for
bisection method
itNew = zeros(1,length(tol2)); % preallocate for iteration vector for
Newton's method

for n = 1:length(tol2)
    [x_finalBis, itBis(n)] = debis(a, b, tol2(n), L1); % compute iterations
for bisection method
    [x_finalNew, itNew(n)] = denewt(x, tol2(n), L1); % compute iterations
for Newton's method
end

```

```
disp(x_finalBis)
disp(x_finalNew)
a = x_finalNew
disp('The smallest root calculated by both the bisection method and newtons
method is 2.0288')
% create a new plot
% Plot the iterations for both methods
figure(2)
semilogx(tol2, itBis, 'b-x', tol2, itNew, 'r-o');
legend('Bisection', 'Newton');
title('Bisection vs Newton');
xlabel('tolerance');
ylabel('# of iterations');
% compute number of iterations required by Bisection then also the amount
% needed for newtons method
```

Competency Functions

Part 1

Part 2

2.0288

2.0288

a =

2.0288

*The smallest root calculated by both the bisection method and newtons method
is 2.0288*

